

**BHASKARACHARYA NATIONAL INSTITUTE FOR  
SPACE APPLICATIONS AND GEO-INFORMATICS**  
WEEKLY PROGRESS REPORT (01/06/2026 – 07/06/2026)

**WEEK 3**

|                               |                                                                                                                                                                                                                                                                                                                                                                         |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Project Name</b>           | OSINT Collection from Social Media Network                                                                                                                                                                                                                                                                                                                              |
| <b>PROJECT DESCRIPTION :</b>  | Continuation of the linkmap OSINT pipeline. Week 3 focused on expanding the frontend with three new pages (Data Explorer, Intelligence Analysis, Scraper Control Panel), building a real-time WebSocket scraping interface, implementing comprehensive security hardening across the full codebase, and resolving all critical bugs identified through deep code audit. |
| <b>GROUP MEMBER :</b>         | Rishav Pal, Sayantan Mandal, Shivam Kumar                                                                                                                                                                                                                                                                                                                               |
| <b>GROUP ID :</b>             | 2026G268                                                                                                                                                                                                                                                                                                                                                                |
| <b>GROUP GUIDE :</b>          | Dr. JHUMMARWALA ABDUL                                                                                                                                                                                                                                                                                                                                                   |
| <b>GROUP COORDINATOR :</b>    | Dr. JHUMMARWALA ABDUL                                                                                                                                                                                                                                                                                                                                                   |
| <b>COLLEGE GUIDE NAME :</b>   | Dr. SHWETA SAHARAN                                                                                                                                                                                                                                                                                                                                                      |
| <b>COLLEGE GUIDE No. :</b>    | 9509676076                                                                                                                                                                                                                                                                                                                                                              |
| <b>COLLEGE GUIDE EMAIL. :</b> | SHWETA.SAHARAN@GSV.AC.IN                                                                                                                                                                                                                                                                                                                                                |
| <b>COLLEGE NAME :</b>         | GATI SHAKTI VISHWAVIDYALAYA                                                                                                                                                                                                                                                                                                                                             |

## WEEK 3 — DAILY WORK LOG

## Day 1–2 | 01–02 June 2026 | Scraper Control Panel &amp; WebSocket Integration

The first two days were dedicated to building the live Scraper Control Panel — a frontend page that allows triggering real-time scraping directly from the browser using WebSocket communication.

- Built backend/scraper\_manager.py — singleton ScraperJob manager with queued sequential execution. ScraperJob dataclass tracks: status, current\_handle, handle\_index, total\_handles, tweets\_collected, tweets\_this\_handle, estimated\_seconds\_remaining, progress\_feed (deque maxlen=50), handle\_previews (top 5 tweets per handle), stop\_requested flag, graph\_ready flag.
- Implemented \_worker\_loop() with asyncio.new\_event\_loop() per job — avoids asyncio.run() conflicts with the FastAPI event loop. Each job runs in its own isolated event loop inside the worker thread.
- Built WebSocket broadcasting via asyncio.run\_coroutine\_threadsafe() with \_on\_done callbacks attached to each future — dead/disconnected clients are detected and removed without blocking. Added \_cleanup\_job\_clients() to release all WS references when a job completes.
- Integrated \_run\_graph\_pipeline() into scraper\_manager — automatically runs GraphTransformer + CypherLoader after each job completes, then sets job.graph\_ready = True and broadcasts a graph\_ready event so the Network page auto-refreshes.
- Built backend/routes/scraper.py — POST /api/scraper/start (validates handles, queues job), POST /api/scraper/stop/{job\_id}, GET /api/scraper/job/{job\_id}, GET /api/scraper/jobs, WebSocket /api/scraper/ws/{job\_id} with clean code=1000 close on completed jobs.
- Built frontend/scraper.html and frontend/js/scraper.js (602 lines) — control panel with handle textarea input, tweet limit slider, green START and red STOP buttons, live status strip (status dot, current handle, tweets collected, ETA), scrolling progress feed with colour-coded type badges, per-handle top-5 tweet preview cards, and job history table.
- Added localStorage-based activeScraperJob tracking so the Network page can detect when a new scrape completes and show a "Refresh Graph" banner automatically.

## Day 3 | 03 June 2026 | Data Explorer &amp; Intelligence Pages

Day 3 built the two remaining new pages: the PostgreSQL-powered Data Explorer and the Connection Intelligence analysis page.

- Built backend/routes/data\_explorer.py (410 lines) — 3 endpoints: GET /data-explorer/tweets (7 simultaneous filters), GET /data-explorer/filter-options (handles, topics, hashtags, mentions, domains), GET /data-explorer/export (rate-limited, up to 5,000 rows). Fixed the double /api prefix bug (routes now correctly prefixed only once via server.py include\_router).
- Filters implemented in data\_explorer: handle (exact), mention (ANY array match), hashtag (ANY array match), keyword (ILIKE full-text), link\_domain (JSONB subquery), date\_from/date\_to (timestampz range), min\_likes, min\_retweets, has\_links (= FALSE fixed from IS FALSE), has\_hashtags, is\_reply, is\_retweet. All sort\_by/order values use \_SAFE\_SORT\_COLS/\_SAFE\_SORT\_DIRS whitelist dict instead of raw f-string interpolation.
- Built frontend/data-explorer.html and frontend/js/data-explorer.js (776 lines) — full filter bar with all 7 filter types, toggle buttons (Has Links, Has Hashtags, Replies Only, Retweets Only, Original Only), active filter pills, result count, paginated tweet table with MENTIONS and DOMAINS extra columns, Blob CSV export.
- Built backend/routes/influence.py (284 lines) — 7 intelligence endpoints: bridge-nodes (cross-cluster connectors), url-coordination (handles sharing same domains), chains (A→B→C mention paths with parameterised depth 2-4), clusters (triangle detection), co-mentions (PostgreSQL unnested pairs), url-spread (domain propagation), anomalies (sudden activity spike detection). All endpoints cached at appropriate TTLs — search endpoints (chains, url-spread) at 5s to prevent stale results for different parameters.
- Built frontend/intelligence.html and frontend/js/intelligence.js (384 lines) — 6 analysis sections: Bridge Nodes table, Co-mention Patterns table, URL Coordination table, Influence Chain Explorer (handle + depth input), Anomaly Flags table, Tight Clusters SVG triangles. All sections load in parallel with skeleton loaders.

## WEEK 3 — DAILY WORK LOG (continued)

## Day 4 | 04 June 2026 | Graph Search on Network Page

Day 4 added a full Graph Search panel to the existing Network page, enabling filtered subgraph exploration directly in the D3.js visualisation.

- Added collapsible GRAPH SEARCH section to frontend/network.html above the metrics strip. Contains: handle input, hashtag input, mention input, link domain input, relationship type multi-select checkboxes (MENTIONED\_BY\_USER, USED\_HASHTAG, SHARED\_LINK, AUTHORED\_ABOUT, BELONGS\_TO, SHARES\_HASHTAG\_WITH), hop depth toggle (1/2/3), min connections number input, SEARCH GRAPH and RESET GRAPH buttons, result count display.
- Updated frontend/js/network.js (1,309 lines) with graph search functions: buildSearchCypher() dynamically constructs parameterised Cypher from filter inputs, applyGraphSearch() fetches filtered nodes/edges from /api/graph/query, re-renders D3 with only matching nodes highlighted (matching nodes brighter + larger radius, non-matching dimmed to opacity 0.15), resetGraphSearch() reloads full graph.
- Added polling auto-refresh: setInterval(listenForGraphUpdates, 5000) checks localStorage for activeScrapperJob every 5 seconds — if graph\_ready event is detected, shows a "New data available — Refresh Graph" banner with blue styling above the canvas.
- Added CSS components: .graph-search, .rel-type-checks, .hop-btn, .graph-refresh-banner to frontend/css/components.css.

## Day 5–6 | 05–06 June 2026 | Comprehensive Security Audit &amp; Bug Fixes

Days 5 and 6 were dedicated to a deep code audit across the entire codebase, identifying 16 bugs across critical, important, and minor severity categories, and implementing fixes.

- CRITICAL: Fixed double /api prefix on data\_explorer routes — all three route decorators corrected from /api/data-explorer/\* to /data-explorer/\* since server.py already adds /api prefix.
- CRITICAL: Fixed asyncio.run() inside daemon thread in scraper\_manager — replaced with asyncio.new\_event\_loop() per job to avoid RuntimeError from conflicting event loops.
- CRITICAL: Fixed IS FALSE SQL bug in data\_explorer — %(has\_links)s IS FALSE always evaluated to True regardless of parameter value. Corrected to %(has\_links)s = FALSE.
- CRITICAL: Fixed f-string SQL injection in data\_explorer.py and tweet\_analysis.py — added \_SAFE\_SORT\_COLS and \_SAFE\_SORT\_DIRS whitelist dicts for ORDER BY clause construction.
- CRITICAL: Fixed graph\_updated race condition — added graph\_ready: bool = False field to ScrapperJob, set to True inside \_run\_graph\_pipeline() on success. Frontend correctly polls data.graph\_ready instead of computing from status+completed\_at.
- CRITICAL: Fixed POSTGRES\_ENABLED not loading in scraper\_manager — added load\_dotenv() for both root .env and backend/.env at module top before PostgresWriter import.
- IMPORTANT: Fixed WebSocket silent failure and memory leak — \_broadcast\_ws() now attaches \_on\_done callback to each future to detect and remove disconnected clients. Added \_cleanup\_job\_clients() called after job completion.
- IMPORTANT: Fixed InMemoryRateLimiter IP memory leak in server.py — added \_maybe\_evict() periodic cleanup that removes IPs not seen for 10 minutes every 5-minute interval.
- IMPORTANT: Fixed WebSocket abrupt close on completed jobs in scraper.py — sends final state snapshot then closes with code=1000 and reason="Job complete".
- IMPORTANT: Fixed cache returning wrong results for different handles in influence/chains — reduced TTL from 60s to 5s on parameterised search endpoints.
- IMPORTANT: Fixed export\_rate\_limits memory leak in data\_explorer — added eviction of entries older than 120 seconds after each export request.
- IMPORTANT: Replaced ReDoS-vulnerable HTML anchor regex with lxml HTMLParser for link extraction in scraper/parser.py — immune to catastrophic backtracking on malformed HTML.
- MINOR: Fixed asyncio.get\_event\_loop() deprecation — replaced with asyncio.get\_running\_loop() in server.py startup. Replaced @app.on\_event with lifespan context manager (FastAPI 0.93+).
- MINOR: Removed \_remove\_dead\_client() dead code from scraper\_manager.py. Fixed async/sync inconsistency in HTML route handlers (removed unnecessary async def).

## Day 7 | 07 June 2026 | PostgreSQL Pool Integration &amp; Final Polish

The final day focused on integrating storage/postgres\_writer.py with the shared connection pool and batch write optimisation in the CLI scrape path.

- Fixed postgres\_writer.py connection churn — replaced `_get_connection()` (raw `psycopg2.connect`) with `_get_pool_conn()` / `_release_conn()` that use the shared `ThreadedConnectionPool` from `postgres_client.py` when available, with a fallback to direct connection for CLI/standalone mode.
- Fixed main.py CLI scrape path — replaced per-tweet `write_tweet()` loop with single `write_tweets_batch()` call per handle. For 100 tweets × 20 handles: reduced from 2,000 sequential DB connections to 20 batch connections.
- Replaced deprecated `@app.on_event("startup"/"shutdown")` with `asyncio` lifespan context manager in `server.py`. Updated `asyncio.get_event_loop()` to `asyncio.get_running_loop()`.
- Completed nav update across all 10 HTML pages to include Data Explorer and Intelligence links with reduced 12px font size and 20px gap to fit all items on one line.
- Verified full pipeline end-to-end: `python main.py --mode frontend`, health endpoint returns `{"neo4j":"Online","postgres":"Online"}`, all 10 pages render correctly.
- Completed final codebase audit confirming zero remaining critical or important bugs.

## WEEK 3 — MEMBER CONTRIBUTIONS

**Rishav Pal — Data Engineer**

- Built the complete scraper backend infrastructure: `scraper_manager.py` (singleton ScraperJob manager, worker thread, WebSocket broadcasting with `_on_done` callbacks, auto graph pipeline), `backend/routes/scraper.py` (5 endpoints including WebSocket), `backend/server.py` lifespan migration.
- Fixed all 6 critical-severity bugs: double API prefix, `asyncio.run()` in thread, IS FALSE SQL, f-string SQL injection, `graph_ready` race condition, `POSTGRES_ENABLED` not loading.
- Fixed `postgres_writer.py` connection churn — implemented `_get_pool_conn()` / `_release_conn()` to route all write operations through the shared `ThreadedConnectionPool`.
- Fixed `main.py` CLI batch write — replaced per-tweet loop with `write_tweets_batch()` reducing 2,000 connections to 20 per full scrape run.
- Replaced `asyncio.get_event_loop()` with `get_running_loop()` and migrated `@app.on_event` to lifespan context manager in `server.py`.
- Fixed `InMemoryRateLimiter` IP memory leak with periodic `_maybe_evict()` cleanup. Fixed `export_rate_limits` memory leak with 120-second entry eviction.

**Sayantana Mandal — OSINT Analyst**

- Built `frontend/scraper.html` and `frontend/js/scraper.js` (602 lines) — complete Scraper Control Panel with WebSocket live feed, handle textarea, tweet limit slider, START/STOP buttons, status strip, progress feed with colour-coded badges, top-5 tweet preview cards, job history table.
- Built `frontend/data-explorer.html` and `frontend/js/data-explorer.js` (776 lines) — full PostgreSQL data exploration interface with 7 filter types, toggle buttons, active filter pills, paginated table with mentions/domains columns, Blob CSV export.
- Built `frontend/intelligence.html` and `frontend/js/intelligence.js` (384 lines) — 6 analysis sections with parallel loading, skeleton loaders, SVG triangle cluster visualization, influence chain explorer with depth selector.
- Updated `frontend/css/nav.css` to reduce font size (12px) and gap (20px) so all 10 nav items fit on one line without wrapping.
- Updated all 10 HTML files nav sections to include Data Explorer (Explorer) and Intelligence links with correct active state detection.
- Verified all frontend pages render correctly and filters produce accurate results against live PostgreSQL and Neo4j data.

**Shivam Kumar — Graph Analyst**

- Built `backend/routes/influence.py` (284 lines) — 7 intelligence analysis endpoints: bridge-nodes (Neo4j cross-cluster connectors), url-coordination (shared domain signals), chains ( $A \rightarrow B \rightarrow C$  mention path traversal), clusters (triangle detection), co-mentions (PostgreSQL unnested pair analysis), url-spread (domain propagation), anomalies (spike detection).
- Built `backend/routes/data_explorer.py` (410 lines) with 7-filter PostgreSQL query builder, `_SAFE_SORT_COLS/_SAFE_SORT_DIRS` whitelist injection prevention, rate-limited export endpoint.
- Added Graph Search panel to `frontend/network.html` and updated `frontend/js/network.js` — `buildSearchCypher()`, `applyGraphSearch()`, `resetGraphSearch()`, `listenForGraphUpdates()` with 5-second polling, graph-refresh-banner auto-notification.
- Replaced ReDoS-vulnerable HTML regex in `scraper/parser.py` with `lxml HTMLParser` — `_extract_hrefs_from_html()` using `etree.fromstring()` with `recover=True`.
- Fixed WebSocket silent failure — `_broadcast_ws()` now uses `add_done_callback()` on each future. Fixed abrupt WS close with `code=1000` graceful close on completed jobs.
- Fixed `influence/chains` cache TTL (60s → 5s) to prevent stale results across different handle searches. Removed `_remove_dead_client()` dead code.

## WEEK 3 — TECHNICAL SUMMARY &amp; BUG AUDIT

## Week 3 Deliverables Summary

| Component                 | Deliverable                                | Status     |
|---------------------------|--------------------------------------------|------------|
| Scraper Control Panel     | WebSocket live scraping from browser       | ✓ Complete |
| scraper_manager.py        | Job queue, worker thread, WS broadcast     | ✓ Complete |
| Auto Graph Update         | Transform + Neo4j load after each scrape   | ✓ Complete |
| Data Explorer Page        | 7 filters, export, mentions/domains cols   | ✓ Complete |
| Intelligence Page         | 7 analysis endpoints + 6 frontend sections | ✓ Complete |
| Graph Search Panel        | Network page filter + live D3 update       | ✓ Complete |
| Bug Audit                 | 16 bugs identified across 3 severities     | ✓ Complete |
| Critical Bugs Fixed       | 6 critical bugs resolved                   | ✓ Complete |
| Important Bugs Fixed      | 7 important bugs resolved                  | ✓ Complete |
| Minor Bugs Fixed          | 3 minor bugs resolved                      | ✓ Complete |
| Postgres Pool Integration | postgres_writer uses shared pool           | ✓ Complete |
| Batch Write Optimisation  | 2,000 → 20 DB connections per scrape       | ✓ Complete |
| lxml HTML Parser          | ReDoS-immune link extraction               | ✓ Complete |
| FastAPI Lifespan          | Replaced deprecated on_event decorators    | ✓ Complete |

## Bug Audit Summary — All 16 Bugs

| #  | Severity    | File               | Issue                                        | Fixed |
|----|-------------|--------------------|----------------------------------------------|-------|
| 1  | ■ Critical  | data_explorer.py   | Double /api prefix → 404 on all calls        | ✓     |
| 2  | ■ Critical  | scraper_manager.py | asyncio.run() in thread → event loop crash   | ✓     |
| 3  | ■ Critical  | data_explorer.py   | IS FALSE SQL → filter never works            | ✓     |
| 4  | ■ Critical  | data_explorer.py   | f-string sort → SQL injection risk           | ✓     |
| 5  | ■ Critical  | scraper_manager.py | graph_updated race condition → always False  | ✓     |
| 6  | ■ Critical  | scraper_manager.py | POSTGRES_ENABLED not loaded → tweets dropped | ✓     |
| 7  | ■ Important | influence.py       | Cypher f-string depth — safety comment added | ✓     |
| 8  | ■ Important | scraper_manager.py | WS silent failure → memory leak              | ✓     |
| 9  | ■ Important | network.js         | Graph update listener runs only once         | ✓     |
| 10 | ■ Important | influence.py       | Cache TTL: wrong result for diff handles     | ✓     |
| 11 | ■ Important | data_explorer.py   | export_rate_limits memory leak               | ✓     |
| 12 | ■ Important | transformer.py     | Stale docstring references merged.json       | ✓     |
| 13 | ■ Important | parser.py          | ReDoS via regex HTML parsing                 | ✓     |
| 14 | ■ Minor     | server.py          | asyncio.get_event_loop() deprecated          | ✓     |
| 15 | ■ Minor     | server.py          | @on_event deprecated → lifespan migration    | ✓     |
| 16 | ■ Minor     | scraper_manager.py | _remove_dead_client() dead code              | ✓     |

## WEEK 3 — DASHBOARD UI MODULES &amp; NOTES

The following notes summarize the final visual and functional state of the linkmap OSINT Intelligence Dashboard after Week 3. The platform now features 10 comprehensive pages running locally over dual PostgreSQL and Neo4j backends.

### Overview Page

Displays live stat counters (handles, tweets, relationships) from the FastAPI backend, an animated SVG network preview with traveling particles, project description, and centralized navigation.

*[UI Note: Presents real-time telemetry metrics and dynamic background animations instead of static assets.]*

### Handle Intelligence Page

Presents all 20 scraped handles equipped with a filter bar (search, min followers, cluster, sort), animated cluster frequency bars, and a paginated sortable table.

*[UI Note: Interactive data tables allow quick sorting, filtering, and frequency bar chart visual profiling for each target.]*

### Influence Network Page — D3.js + Graph Search

Features a Neo4j-style dark canvas highlighting coloured nodes (purple=User, green=Hashtag, orange=Link, blue=Topic). Incorporates a new Graph Search panel with handle/hashtag/domain filters, relationship toggles, and live graph reflow capabilities.

*[UI Note: Renders a fully interactive and draggable D3.js force-directed graph visualization responsive to search filters.]*

### Hashtag Analysis Page

Aggregates top hashtag frequency bars from PostgreSQL, tracking co-occurrence pairs and burst campaign signals. Includes a comprehensive filter bar (hashtag search, min uses, topic categorization).

*[UI Note: Utilizes horizontal bar charts for hashtag volume alongside an alert section for anomalous burst activity.]*

### Topic Clusters Page

Breaks down 12 topic clusters detailing tweet counts, handle counts, and top hashtags per cluster. Contains a sidebar vertical distribution chart and expandable data rows.

*[UI Note: Features vertical distribution progress bars alongside collapsable tables for drill-down topic analysis.]*

### Campaign Detection Page

Monitors for coordinated activity via alert banners, mapping burst events via a hashtag cluster radial hub SVG visualization and a co-occurrence pattern table.

*[UI Note: Relies heavily on a central circular/radial node graph highlighting bot-like or coordinated account clusters.]*

### Tweet Analysis Page

A robust PostgreSQL tweet search interface with 6 multi-select filters. Results populate a paginated table enhanced with entity type badges, hashtag pills, topic labels, and integrated SVG charts.

*[UI Note: Acts as a comprehensive datagrid offering inline trend lines (Tweets by Day) and direct CSV export buttons.]*

### Final Architecture — Week 3 State

| Layer          | Technology                 | Purpose                                          |
|----------------|----------------------------|--------------------------------------------------|
| Scraping       | Playwright + Chromium      | Anti-ban browser automation, X/Twitter scraping  |
| Entity Extract | Python regex + lxml        | Hashtags, mentions, links (ReDoS-immune)         |
| Raw Storage    | PostgreSQL (psycopg2 pool) | All tweet + profile data, 7-filter query support |
| Graph DB       | Neo4j Desktop (Cypher)     | Relationship graph: User, Hashtag, Link, Topic   |
| Transform      | graph/transformer.py       | PostgreSQL → Neo4j CSV files (no local JSON)     |
| API Backend    | FastAPI + WebSocket        | 10 route files, 40+ endpoints, lifespan managed  |

|              |                             |                                                 |
|--------------|-----------------------------|-------------------------------------------------|
| Frontend     | Vanilla JS (ES modules)     | 10 pages, D3.js graph, SVG charts, no framework |
| Real-time    | WebSocket (starlette)       | Live scrape progress, graph auto-refresh signal |
| Intelligence | Neo4j Cypher + Postgres SQL | Bridge nodes, chains, clusters, anomalies       |

---

*Report prepared by: Rishav Pal (Data Engineer), Sayantan Mandal (OSINT Analyst), Shivam Kumar (Graph Analyst) | Week 3: 01 June – 07 June 2026*